

Módulo 7

Seguridad y optimización de costos en DevOps

DevSecOps · SAST · DAST · FinOps

Clase 7 · Duración 2 h

```
sast

$ semgrep --config auto

3 findings:
! sql-injection
! hardcoded-secret
! weak-hash

✗ build bloqueado
```

Dónde estamos en el bucle

Hoy no agregamos una fase nueva: agregamos dos capas que atraviesan todas las fases



Las dos preguntas de hoy

¿Cómo metés seguridad en un pipeline que despliega veinte veces por día, sin convertirte en el cuello de botella?

Y cuando la infraestructura se crea sola con Terraform y escala sola con Kubernetes, ¿quién se da cuenta de que la factura se triplicó?

Agenda de hoy

Dos horas, dos temas grandes y un break de 10 minutos

1

Por qué DevSecOps

El modelo viejo, el costo de encontrar tarde y el shift left

20 min

2

SAST y DAST

Qué ve cada uno, qué no ve ninguno, dónde van en el pipeline

30 min



Break

Estiren las piernas

10 min

3

FinOps

Tagging, rightsizing, spot, autoscaling y métricas

30 min

4

Integrar todo + demo

El pipeline completo con seguridad y control de costos

20 min

Al terminar la clase vas a poder...

1

Explicar qué es DevSecOps y por qué "shift left"

y por qué dejar la seguridad para el final sale carísimo

2

Distinguir SAST de DAST con criterio propio

qué encuentra cada uno, cuándo corre y por qué son complementarios

3

Nombrar lo que ni SAST ni DAST ven

dependencias, secretos, imágenes e infraestructura como código

4

Aplicar las cuatro estrategias base de FinOps

tagging, rightsizing, instancias spot y autoscaling por costo

5

Decidir cuándo romper el build por seguridad

que es la decisión más política de todo el pipeline

Los objetivos 2 y 4 son los que evalúa la plataforma. El 5 es el que más vale en el trabajo real.

01

BLOQUE 1 · 20 MINUTOS

Por qué DevSecOps

La seguridad dejó de ser una auditoría al final para volverse una propiedad del pipeline.

El segundo muro

Durante años la seguridad fue una puerta al final del camino, no parte del camino

Diseño

Desarrollo

Testing

Pre-release

AUDITORÍA
DE SEGURIDAD

CÓMO FUNCIONABA

- El equipo construía durante meses
- Al final, seguridad auditaba todo junto
- Aparecía un informe de 80 páginas
- Había que rehacer decisiones de arquitectura
- El release se atrasaba semanas

POR QUÉ DEJÓ DE FUNCIONAR

- Si desplegás 20 veces por día, no hay auditoría que alcance
- Seguridad pasó a ser el cuello de botella del negocio
- Los equipos aprendieron a esquivarla
- Encontrar un problema al final cuesta mucho más que al principio

El costo de encontrar tarde

Arreglar la misma vulnerabilidad cuesta distinto según cuándo la encontrás

Mientras escribís el código

Minutos. Todavía tenés el contexto en la cabeza.



1×

En el pull request

Horas. Alguien más lo revisa, se corrige, se vuelve a aprobar.



~5×

En testing

Días. Hay que reabrir el ticket, retestear, recoordinar.



~15×

Ya en producción

Semanas, más el incidente. Y a veces, la multa y la noticia.



~100×

De ahí viene "shift left": correr los controles lo más a la izquierda posible del pipeline, donde todavía son baratos.

Qué significa "shift left" en la práctica

No es una herramienta: es mover cada control al momento más temprano en que puede correr

En tu editor	Linters de seguridad y detección de secretos mientras escribís	Segundos
Al hacer commit	Hook que bloquea si estás por subir una credencial	Segundos
En el pull request	SAST sobre el código y escaneo de dependencias	Minutos
Al construir la imagen	Escaneo de la imagen Docker y de los manifiestos de IaC	Minutos
En staging, ya desplegado	DAST contra la aplicación corriendo	~10 minutos
En producción, continuo	Monitoreo de nuevas CVE sobre lo que ya desplegaste	Permanente

Fijate que no hay una etapa "de seguridad": hay controles de seguridad adentro de cada etapa que ya existía.

DevSecOps es CALMS aplicado a la seguridad

Las mismas cinco ideas del módulo 1, con otro sujeto



Cultura

La seguridad es responsabilidad de todos, no de un equipo que dice que no



Automatización

Los controles corren solos en el pipeline, no en una reunión



Lean

Pocos controles bien elegidos; cada falso positivo es desperdicio



Medición

Cuántas vulnerabilidades, de qué severidad, cuánto tardamos en cerrarlas



Compartir

Seguridad enseña y da herramientas; no audita y castiga

Si el equipo de seguridad solo aparece para bloquear, el equipo de desarrollo va a aprender a no llamarlo.

02

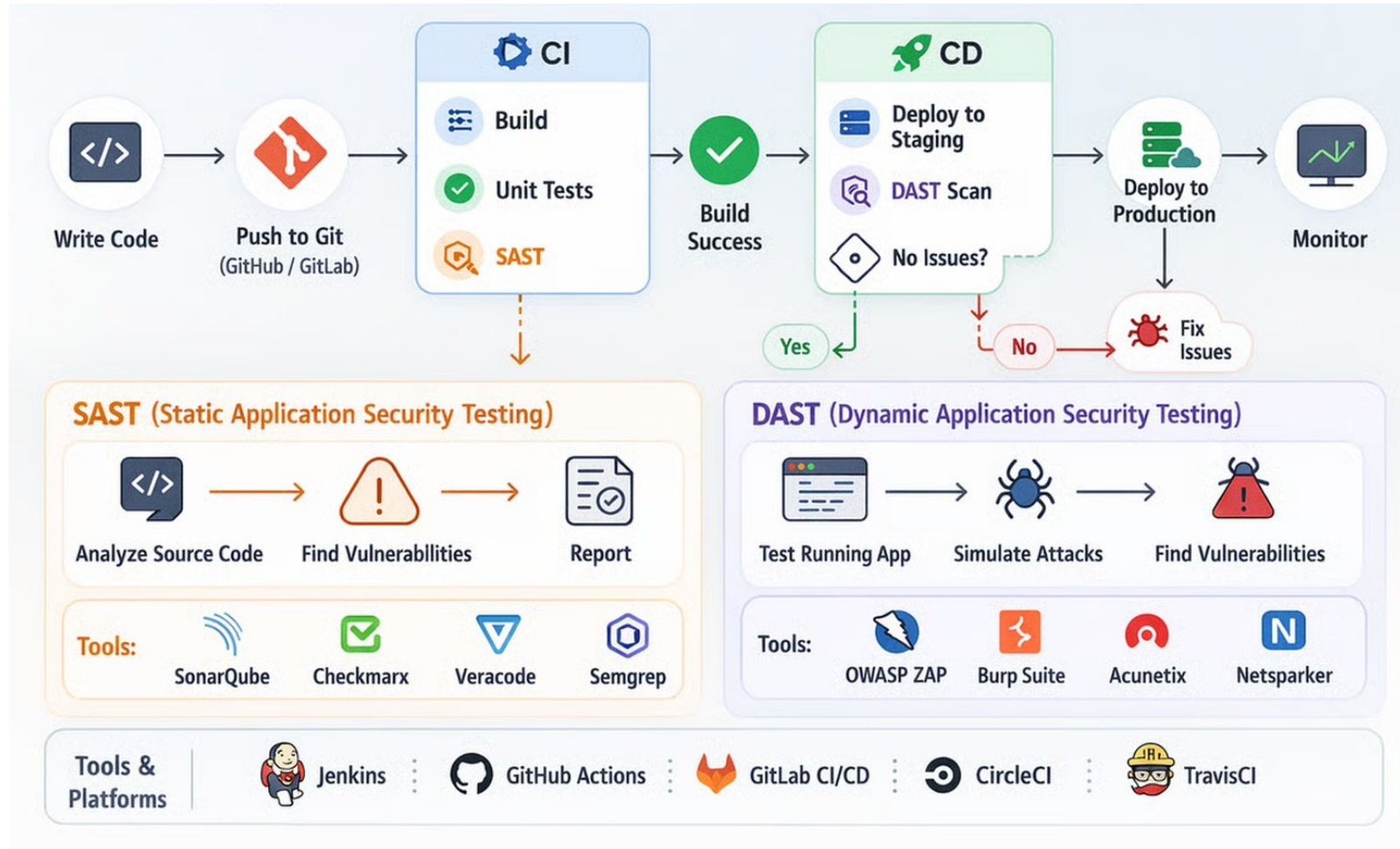
BLOQUE 2 · 30 MINUTOS

SAST y DAST

Dos formas de buscar vulnerabilidades: leyendo el código, o atacando la aplicación.

El pipeline completo con seguridad

SAST corre adentro de CI; DAST corre después del despliegue a staging



Fijate dónde está cada uno: SAST antes de construir, DAST después de desplegar. Esa posición es toda la diferencia entre los dos.

SAST — Static Application Security Testing

Lee tu código fuente sin ejecutarlo, buscando patrones peligrosos

QUÉ ENCUESTRA BIEN

- Inyección SQL por concatenar strings
- Credenciales escritas en el código
- Algoritmos de hash obsoletos
- Validación de entrada que falta

QUÉ NO PUEDE VER

- Errores de configuración del servidor
- Fallas que solo aparecen en ejecución
- Problemas de autenticación entre servicios
- Lo que pasa con datos reales de usuarios

ejemplo real de un hallazgo

```
# Lo que SAST marca en rojo:  
query = "SELECT * FROM users WHERE id = " + user_input  
  
# Lo que espera ver:  
query = "SELECT * FROM users WHERE id = %s", (user_input,)
```

Herramientas

SonarQube · Semgrep
Checkmarx · Veracode
CodeQL · Bandit

DAST — Dynamic Application Security Testing

Ataca tu aplicación corriendo, como lo haría alguien de afuera

QUÉ ENCUENTRA BIEN

- Cabeceras de seguridad que faltan
- Cookies mal configuradas
- Endpoints expuestos sin autenticación
- Errores que filtran información interna

QUÉ NO PUEDE VER

- En qué línea de código está el problema
- Caminos del código que no llegó a recorrer
- Nada, si la app no arranca o no es alcanzable
- Lógica de negocio mal pensada

salida real de OWASP ZAP

```
WARN-NEW: Missing Anti-clickjacking Header [10020]
WARN-NEW: Content-Security-Policy Header Not Set [10038]
WARN-NEW: Cookie No HttpOnly Flag [10010]

FAIL-NEW: 0   WARN-NEW: 3   PASS: 48
```

Herramientas

OWASP ZAP · Burp Suite
Nikto · Acunetix
Netsparker

SAST vs DAST, lado a lado

La tabla que conviene que anoten: es lo que evalúa la plataforma

DIMENSIÓN	SAST	DAST
Qué analiza	El código fuente, sin ejecutarlo	La aplicación en ejecución
Tipo de prueba	Caja blanca: ve todo el código	Caja negra: no conoce el código
Cuándo corre	En CI, en cada commit o PR	En CD, después de desplegar a staging
Velocidad	Segundos o minutos	Minutos u horas
Qué te dice	Archivo y línea exacta	La URL y el request que falló
Falsos positivos	Muchos: ve código que quizá nunca corre	Pocos: lo que encuentra es real
Falsos negativos	Pocos en lo que sabe buscar	Muchos: solo ve lo que logró recorrer
Necesita la app funcionando	No	Sí, desplegada y alcanzable

No se elige uno: SAST te dice dónde está el bug, DAST te confirma que es explotable.

Lo que ni SAST ni DAST ven

El código propio es apenas una parte de lo que desplegarás

SCA

Software Composition Analysis

Revisa tus dependencias contra bases de vulnerabilidades conocidas. El 80 % de tu aplicación es código de terceros.

Trivy · Snyk · Dependabot

Secretos

Secret scanning

Busca claves de API, tokens y contraseñas en el repositorio y en todo el historial de Git.

gitleaks · TruffleHog

Imágenes

Container scanning

Escanea la imagen Docker: el sistema operativo base y las librerías que trae instaladas.

Trivy · Grype · Clair

IaC

Infrastructure as Code scanning

Revisa tu Terraform y tus manifiestos de Kubernetes: puertos abiertos, buckets públicos, permisos de más.

Checkov · tfsec · Kubescape

El dato del SCA es el que más impacta: la mayor parte del código que desplegarás no lo escribiste vos.

¿Rompeamos el build o no?

La decisión más política del pipeline, y la que define si el programa sobrevive

SI BLOQUEÁS POR TODO

- El pipeline se rompe diez veces por día
- Los equipos piden excepciones permanentes
- En dos semanas alguien desactiva el escáner

SI NO BLOQUEÁS NUNCA

- El reporte se acumula y nadie lo lee
- La deuda de seguridad crece sin techo
- El escáner existe para la auditoría, no para proteger

La política que funciona en la práctica

CRÍTICA y ALTA

Rompen el build. No se mergea.

MEDIA

No rompe, pero crea un ticket con fecha de vencimiento.

BAJA e INFORMATIVA

Queda en el reporte. Se revisa en bloque cada tanto.

ACTIVIDAD 1 · 10 MINUTOS

¿SAST, DAST o ninguno?

Para cada hallazgo, decí qué herramienta lo habría detectado. Respondé por el chat.

1 Una contraseña de base de datos escrita directamente en un archivo .py

¿?

2 Falta la cabecera Content-Security-Policy en todas las respuestas HTTP

¿?

3 La librería express que usás tiene una vulnerabilidad crítica publicada

¿?

4 Una consulta SQL armada concatenando lo que escribe el usuario

¿?

5 Un endpoint /admin que responde sin pedir autenticación

¿?

6 Un bucket de S3 declarado como público en el archivo de Terraform

¿?

Ojo: dos de los seis no los detecta ni SAST ni DAST. Ahí está la gracia del ejercicio.

Break

10 minutos

Volvemos con FinOps: cómo no fundirte en la nube

03

BLOQUE 3 · 30 MINUTOS

FinOps

La nube no tiene techo. La factura tampoco, salvo que alguien se ocupe.

Por qué la factura se descontrola

Todo lo que hace buena a la nube la hace cara si nadie mira



Crear recursos es trivial

Una línea de Terraform crea un cluster. Borrarlo requiere acordarse.



Nadie siente el gasto

La tarjeta es de la empresa. El que crea el recurso no ve la factura.



El autoscaling escala hacia arriba

Y muchas veces nadie configuró que vuelva a bajar.



Los ambientes de prueba no se apagan

Staging corre sábado y domingo a las tres de la mañana, sin nadie usándolo.



El sobredimensionamiento parece gratis

"Pongámosle 16 GB por las dudas" cuesta todos los meses, para siempre.



Los datos salientes se cobran

La transferencia hacia afuera suele ser la línea sorpresa de la factura.

Dato de la industria: se estima que cerca de un tercio del gasto cloud se desperdicia en recursos ociosos o sobredimensionados.

Qué es FinOps

No es "gastar menos": es que cada peso gastado sea una decisión consciente

Una práctica cultural donde ingeniería, finanzas y negocio comparten la responsabilidad sobre el gasto cloud, con datos en tiempo real y decisiones basadas en el valor que ese gasto genera.

INFORMAR

Visibilidad

Saber cuánto se gasta, en qué y quién lo generó. Sin esto no se puede hacer nada más.

Tagging, dashboards, reportes por equipo

OPTIMIZAR

Acción

Reducir lo que no aporta valor: apagar lo ocioso, ajustar lo sobredimensionado, usar precios más baratos.

Rightsizing, spot, autoscaling, reservas

OPERAR

Continuidad

Convertirlo en hábito: alertas de presupuesto, revisión periódica, responsabilidad por equipo.

Políticas, alertas, revisión mensual

Es un ciclo, no una escalera: cuando terminás de operar, volvés a informar con datos nuevos.

1 · Tagging — sin esto, nada de lo demás funciona

Etiquetar cada recurso con metadatos que digan de quién es y para qué

```
etiquetas en Terraform

resource "aws_instance" "api" {
  instance_type = "t3.medium"

  tags = {
    Proyecto    = "AppWeb"
    Ambiente    = "produccion"
    Equipo      = "pagos"
    CentroCosto = "CC-3041"
  }
}
```

Qué te habilita

- Saber cuánto cuesta cada proyecto
- Cobrarle el gasto a cada equipo
- Encontrar recursos huérfanos
- Apagar por ambiente automáticamente

La regla que hace que funcione

Las etiquetas tienen que ser obligatorias desde el código, no un acuerdo de buena voluntad. Se valida en el pipeline con un escáner de IaC: si un recurso de Terraform no trae las etiquetas obligatorias, el pipeline no lo aplica.

Así es como seguridad y costos terminan usando exactamente el mismo mecanismo: una validación automática en el pipeline.

2 · Rightsizing — pagar por lo que usás

Ajustar el tamaño del recurso a la demanda real, medida, no imaginada

RECURSO	ASIGNADO	USO REAL (p95)	ACCIÓN
api-produccion	8 vCPU · 32 GB	1,8 vCPU · 6 GB	Bajar a 2 vCPU · 8 GB
worker-batch	4 vCPU · 16 GB	3,9 vCPU · 15 GB	Dejar como está
db-staging	4 vCPU · 16 GB	0,2 vCPU · 2 GB	Bajar y apagar de noche
cache-redis	2 vCPU · 8 GB	0,4 vCPU · 7,2 GB	Bajar vCPU, mantener RAM

Por qué p95 y no promedio

El promedio te hace recortar de más y romper los picos. El p95 te dice cuánto necesitás el 95 % del tiempo. Es la misma lección del módulo 6.

Hacelo gradual

Bajá un escalón, observá una semana con el monitoreo del módulo 6, y recién después bajá otro. Rightsizing sin observabilidad es adivinar.

3 · Instancias spot — capacidad sobrante a precio de remate

El proveedor te alquila lo que le sobra, con la condición de poder quitártelo

CÓMO FUNCIONA

- Descuentos típicos del 60 % al 90 %
- El proveedor puede interrumpirla con 2 minutos de aviso
- Es la misma máquina: no es más lenta ni peor

SÍ, USALAS

- Runners de CI/CD
- Procesos batch y ETL nocturnos
- Entrenamiento de modelos con checkpoints
- Workers que leen de una cola

LA PREGUNTA CORRECTA

¿Qué pasa si esta máquina desaparece en dos minutos y sin aviso previo? Si la respuesta es "se reintenta solo y nadie se entera", podés usar spot.

NO, NI SE TE OCURRA

- Bases de datos de producción
- El único nodo de un servicio crítico
- Procesos largos sin posibilidad de reanudar
- Cualquier cosa con estado en disco local

4 · Autoscaling con enfoque en costos

Escalar hacia arriba es fácil de justificar. Lo que ahorra plata es escalar hacia abajo.

1

Definí el mínimo con honestidad

¿Cuántas réplicas necesitás a las 4 de la mañana de un domingo? Probablemente no sean las mismas que a las 11 de un martes.

2

Apagá los ambientes que no son producción

Staging y desarrollo apagados de 20 a 8 y los fines de semana recortan cerca del 70 % de las horas de esos ambientes.

3

Escalá por la métrica correcta

Si tu cuello de botella es la cola de mensajes, escalar por CPU no te sirve: vas a tener diez pods ociosos esperando.

4

Cuidado con el ida y vuelta

Un autoscaler mal configurado prende y apaga cada dos minutos. Eso cuesta más que dejarlo quieto.

La cuenta rápida: un ambiente de staging apagado nueve horas por día y los fines de semana cuesta un tercio de lo que costaba.

Las métricas que hacen falta

Igual que en observabilidad: sin medición, todo lo anterior es opinión

Costo por etiqueta

Cuánto gasta cada proyecto, equipo o ambiente

Es la métrica que habilita la conversación con cada equipo

Utilización

Qué porcentaje de lo que pagás se usa realmente

Por debajo del 20 % sostenido, hay rightsizing para hacer

Ahorro por spot

Cuánto dejaste de gastar usando capacidad interrumpible

Sirve para justificar el esfuerzo de adoptar spot

Costo unitario

Cuánto cuesta atender un pedido, un usuario o una transacción

La única métrica que relaciona el gasto con el valor del negocio

El costo unitario es el que te deja decir: "gastamos el doble, pero atendemos cuatro veces más pedidos". Sin él, todo aumento parece un problema.

04

BLOQUE 4 · 20 MINUTOS

Todo junto en el pipeline

Seguridad y costos no son dos temas: son dos validaciones automáticas en el mismo lugar.

El pipeline del proyecto final

Cada etapa gana una validación automática de seguridad o de costo

1	Commit	Secret scanning: ninguna credencial entra al repo
2	Build	SAST sobre el código + SCA sobre las dependencias
3	Package	Escaneo de la imagen Docker antes de publicarla
4	IaC	Checkov sobre Terraform: permisos, puertos y etiquetas obligatorias
5	Deploy a staging	DAST con OWASP ZAP contra la app ya corriendo
6	Producción	Monitoreo del módulo 6 + alertas de presupuesto

Ninguna de estas seis validaciones la ejecuta una persona. Todas las dispara el mismo push que ya hacías.

DEMO EN VIVO · 12 MINUTOS

Encontrar vulnerabilidades reales

Sobre una app con fallas puestas a propósito, para ver qué detecta cada herramienta.

● ● ● analisis estatico

```
# SAST — lee el código
docker run --rm -v $PWD:/src \
  semgrep/semgrep semgrep \
  --config auto /src

# SCA + imagen + secretos
trivy fs .
trivy image practica:v1
```

● ● ● analisis dinamico

```
# DAST — ataca la app viva
docker compose up -d

docker run --rm --network host \
  zaproxy/zap-stable \
  zap-baseline.py \
  -t http://localhost:8000
```

El momento clave: la misma app, dos herramientas, hallazgos completamente distintos. Ahí se entiende por qué hacen falta las dos.

Actividad 2: ¿dónde está la plata?

Un equipo gasta 4.200 USD por mes. Decidí qué harías primero y por qué.

Cluster de producción	1.800 USD	Utilización promedio del 65 %, picos del 90 %
Cluster de staging	900 USD	Prendido 24/7. Se usa de lunes a viernes, de 9 a 18
Base de datos de producción	700 USD	16 vCPU asignadas, p95 de uso: 3 vCPU
Runners de CI/CD	500 USD	Instancias bajo demanda, trabajo 100 % reintentable
Transferencia de datos	200 USD	Sin etiquetas. Nadie sabe qué servicio la genera
Recursos sin etiquetar	100 USD	Nadie sabe de quién son ni si se usan

Pensalo en orden de ahorro por esfuerzo: ¿qué harías el lunes a la mañana y qué dejarías para el trimestre que viene?

Para la próxima clase

La última antes del proyecto final

1

Escanear tu repo

- Correr SAST, SCA y escaneo de imagen sobre tu proyecto
- Correr DAST contra la app levantada con docker compose
- Agregar los escaneos al workflow de GitHub Actions
- Arreglar al menos un hallazgo y documentarlo

2

Plan de FinOps

- Tomá el caso de la Actividad 2 (o uno real que conozcas)
- Priorizá tres acciones por ahorro sobre esfuerzo
- Estimá el ahorro mensual de cada una, con la cuenta hecha
- Definí qué métrica usarías para verificar que funcionó

Próxima clase — Módulo 8: Proyecto final, pipeline CI/CD integral

Todo el curso en un entregable: Docker, Terraform, Kubernetes, CI/CD, observabilidad, seguridad y costos.

Lo que nos llevamos

1 Encontrar una vulnerabilidad en producción cuesta cien veces más que encontrarla al escribir el código

2 SAST lee el código sin ejecutarlo; DAST ataca la app corriendo sin conocer el código

3 Ninguno de los dos ve tus dependencias, tus secretos, tus imágenes ni tu infraestructura como código

4 Bloqueá el build solo por severidad crítica y alta: si bloqueás por todo, te desactivan el escáner

5 FinOps empieza por etiquetar: sin saber de quién es cada recurso, no podés optimizar nada

¿Preguntas?